Prompt - Patch Relationship Prediction

First, analyze the following two relationships for commits in the patch group:

---

① ***Merge***: A patch consolidates the effects of multiple related patches on separate development lines. Code changes should satisfy the condition that *Merge patch* codes are composed of *Merged patches* codes. Note that distinguish between *Merge patch* (the parent commit integrating changes) and *Merged patches* (original changes being integrated).
[*Output Schema*] $R_{ID\text{-}\text{“}Merge\text{”}}$: {*Merge_patch*: [*Merged_patch*1, *Merged_patch*2, ...], ...}

---

② ***Mirror***: Patches with identical/similar fixes for different software versions (e.g., *Ver* 4.1, *Ver* 5.2, ...) or branches (e.g., Br *main*, Br *release-X.Y*, ...). Commit messages and code changes should show great similarity.
[*Output Schema*] $R_{ID\text{-}\text{“}Mirror\text{”}}$: {*PrimaryVer_patch*: [*Ver*1_*patch*, *Ver*2_*patch*, ...], ...}, {*MainBr_patch*: [*Br*1_*patch*, *Br*2_*patch*, ...], ...}

Then, identify the following three relationships further. Note that the dependency of patches is the foundation for forming these relationships. A dependency between two patches means:

- The modifications in the subsequent patch need to be based on the code of the previous patch.
- The subsequent patch needs to use the newly added or modified functions, variables, etc. from the previous patch.
- The order of the two patches can affect the logic or functionality of the code and the fixing effectiveness.

---

③ ***Better Solution***: A following patch 1) aims to improve the readability, running efficiency, etc. of the previous patch without affecting security, or 2) only modifies text information (e.g., comments) without code changing, or 3) can replace the refined part of the previous patch's vulnerability-fixing logic. Messages of following patches are often obtained by making slight modifications to messages of the previous patch. Code changes should satisfy the condition that a following patch deletes some codes from the previous patch and replace them with codes of the same security level, or just adds comments, etc.
[*Output Schema*] $R_{ID\text{-}\text{“}Better\ Solution\text{”}}$: {*Original_patch*: [*Better_patch*, ...], ...}

---

④ ***Fixing-of-Fixing***: A following patch 1) fixes the issues introduced by the previous patch, such as incorrect fixing logic, inappropriate function or variable names, or regular expression errors, etc., or 2) backports the code to the previous version without errors, or 3) deletes incorrect calls or definitions. Distinguish it from that the subsequent patch fixes the parts that were not resolved in the previous patch.
[*Output Schema*] $R_{ID\text{-}\text{“}Fixing\text{-}of\text{-}Fixing\text{”}}$: {*Flawed_patch*: [*Corrected_patch*, ...], ...}

---

⑤ ***Collaboration***: The previous patch 1) cannot completely solve all the problems, or 2) prepares for the fixes in the subsequent patch such as defining functions. All patches are required for complete vulnerability remediation.
[*Output Schema*] $R_{ID\text{-}\text{“}Collaboration\text{”}}$: {*Previous_patch*: [*Subsequent_patch*, ...], ...}

Last, if there is no dependency between two patches, consider the following relationship:

---

⑥ ***Separation***: Patches contribute to remediation without dependency, their application order does not affect fixing effectiveness. The patches 1) address different aspects of the vulnerability independently, or 2) perform fixes at multiple code locations where the vulnerable pattern is replicated.
[*Output Schema*] $R_{ID\text{-}\text{“}Separation\text{”}}$: {*Patch*1}, $R_{ID\text{-}\text{“}Separation\text{”}}$: {*Patch*2}, ...

Fig. 1: The prompt used in Phase-3